SoulTalk

시연 영상
https://gqr.sh/2eFZ

다양한 성격을 가진 페르소나 AI와 감정 기반 채팅·영상통화 서비스



에너지가 넘치고,
주변을 밝게 만드는
써니 (Sunny)



까칠하고 예민한
완벽주의자
에바 (Eva)



다혈질적이지만
리더십있는
로건 (Logan)



소심하지만
따뜻한 용기를 가진
제이미 (Jamie)



매사에 무심하고
무던한 성격의
리나 (Lena)

플랫폼 android
개발 인원 3명
개발 기간 2024.10 ~ 2025.09 (12개월)
담당 역할 LLM·백엔드·인프라 담당 (기여도 45%)

- Persona기반 LLM 프롬프트 엔지니어링 및 성능 최적화
- 요약·체크포인팅 모델을 활용한 멀티턴 대화 맥락 유지
- GraphRAG 기반 AI 선톡 기능 개발
- 실시간 End-to-End AI파이프라인(TTS·Unreal·A2F) 아키텍처 설계 및 구축
- 온프레미스 클라우드(AWS) 혼합 서비스 인프라 구축 및 배포 자동화





주요 사용 기술

감정인식/OnDevice AI
- MobileNetV3 파인튜닝
- KoELECTRA-small v3 파인튜닝

AI/머신러닝 프레임워크
- HuggingFace, PyTorch, TensorFlow Lite

인프라
- AWS, Docker

LLM 응답 생성
- LangChain, OpenAI API, GraphRAG

맥락유지
- LangGraph. bart모델 기반 요약

3D 애니메이션/렌더링
- NVIDIA Audio2Face 3d NIM
- Unreal Engine

SoulTalk | 전체 서비스 구조

OnDevice AI

감정분석AI모델

TensorFlow Lite

- Text 분석모델 (오늘 날씨가...)
- Voice 분석모델
- Face 분석모델

SoulTalk
멀티모달 후기퓨전 알고리즘

Google Cloud Speech API

Mediapipe
얼굴탐지

FCM

유저관리 API 서버

JWT
AWS EC2
FastAPI

S3
AI 모델 배포
언리얼엔진 인스턴스 배포

Database 서버

- ElastiCache — Redis
- RDS — MySQL
- EC2 — neo4j

채팅/영상통화

로드 밸런서

Auto Scaling 그룹
CloudWatch

Socker 서버 (AWS EC2)
Express
JWT
Socket.IO

Socker 서버 (AWS EC2)
Express
JWT
Socket.IO

미디어 서버

AWS EC2
Coturn
STUN/TURN 서버

얼굴 애니메이션 생성 서버

- ON-PREMISE — Nvidia Audio2Face-3D
- AWS EC2 — Nvidia Audio2Face-3D

AI 응답 생성 서버

AWS EC2

FastAPI
API 형태로 서빙

(1) AI 감정 산출

PyTorch

- (비)유해 표현 분류 모델
- 대화 주제 변경 탐지 모델
- 텍스트 감성 탐지 모델

VAD
SoulTalk
페르소나 감정 변화 산출 알고리즘

(2) 대화 맥락유지 시스템

PyTorch
- 대화 요약 모델
- SQLite — 대화 상태 관리
- LangGraph — 멀티턴 대화이력 요약본 생성

(3) 프롬프트 생성

LangChain

페르소나 주입
+
생성된 AI의 감정
+
대화이력 요약본

(4) LLM 응답 생성 API

OpenAI
gpt-4.1 api

(5) TTS API 요청

OpenAI
gpt-4o-mini-tts api

gRPC

Unreal Engine 서버

ON-PREMISE / AWS EC2

UNREAL ENGINE
WebRTC
PixelStreaming WebServer
node
PixelStreaming Signaling Server

SoulTalk | AI와 채팅/영상통화 End-To-End 파이프라인 구축

OnDeviceAI 감정인식 → **AI 감정 생성 & LLM 응답 생성 (공통)** → **TTS 생성 및 스트리밍** → **A2F 얼굴 애니메이션 생성**

영상통화

(1) 사용자 입력 전처리

- 오디오
  - STT → 텍스트 변환
  - 멜스펙트로그램 이미지 변환
- 비디오
  - 얼굴 탐지 및 전처리

(2) 각 모달리티별 감정분석

- 텍스트
- 음성
- 표정

(3) 감정분석 결과값 퓨전

채팅

(1) 사용자 텍스트 입력

(2) Text 감정인식 모델로 분석

(1) 페르소나별 기본값 선택



각 **페르소나별 성격, 나이, 취향, 기본 감정 등이 다름**
사용자의 응답에 따른 **감정변화 양상 및 응답 내용이 달라짐**

(2) AI캐릭터의 현재 감정 계산

(AI감정변화 산출 알고리즘 적용)

(3) 유저와의 대화이력 요약 및 맥락유지 처리

(4) 캐릭터별 프롬프트 생성

- 페르소나 정보
- AI캐릭터의 감정
- 대화이력 요약본

(5) OpenAI API로 응답 출력

영상통화

(1) TTS (Text-to-Speech) 실시간 음성 생성

- 페르소나별 목소리 & 발화스타일 지정
- OpenAI TTS(Text-to-Speech) : 청크단위 실시간 청크 스트리밍 반환

(2) gRPC 기반 실시간 오디오 청크 중계

- Python 서버 → UnrealEngine 인스턴스 (C++ 클라이언트)

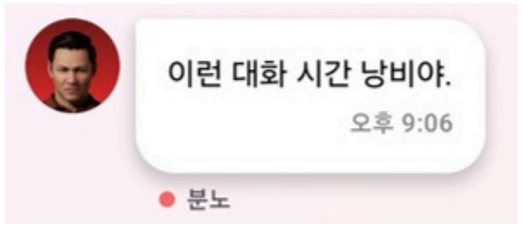
(3) Audio2Face 실시간 표정 애니메이션 생성

(4) 애니메이션 & 오디오 렌더링



채팅

응답 출력



SoulTalk | OpenAI TTS 와 Audio2Face를 활용한 실시간 얼굴 애니메이션

입력
대화할 AI 페르소나 : **써니**
표현할 AI 감정 : **Inspired**
생성된 LLM 응답 Text : " ~~~~"

(1) 페르소나별 목소리 선택

써니	coral (밝고 친근한 여성 목소리)
에바	sage (날카롭고 예민한 여성 목소리)
로건	ash (중후한 남성 목소리)
제이미	fable (조용하고 차분한 남성 목소리)
리나	shimmer (담백하고 직설적인 여성 목소리)

(2) 페르소나별 감정 지시문을 통한 발화스타일 지정

페르소나별 "Inspired" 감정 지시문

써니	사용자 메시지에 깊이 감동받은 감정을 담아 응답.
에바	짧고 강한 감탄, 그리고 까칠한 피드백이 섞인 반말
로건	'괜찮네, 인정한다', '생각보다 인상 깊었어'처럼 간결하게 감동을 표현
제이미	감정을 크게 드러내진 않지만, 조용히 감탄하거나 말끝을 흐려서 감동이 전해지게
리나	감탄은 짧게만 하고, 과한 감정 없이 인상 깊었다는 느낌만 솔직하게

OpenAI
gpt-4o-mini-tts
api

(3) 청크단위 실시간 TTS 요청

(4) 구독중인 언리얼엔진 인스턴스에 청크단위 TTS 중계

gRPC 중계 서버
(Python)

구독 요청

AudioChunk

AudioChunk

(5) 오디오청크를 페르소나별 Audio2Face 서버로 전송

언리얼엔진
인스턴스

gRPC 클라이언트
(C++)

AudioChunk

A2F 플러그인

AudioChunk
+
표정 애니메이션
BlendShape

메타휴먼 캐릭터
Actor

(7) 생성된 애니메이션 데이터 및 오디오 렌더링

Audio
Chunk

(6) 얼굴 애니메이션 생성

AudioChunk
+
표정 애니메이션
BlendShape

Nvidia
Audio2Face-3D

Nvidia
Audio2Face-3D

Nvidia
Audio2Face-3D

Nvidia
Audio2Face-3D

Nvidia
Audio2Face-3D

써니
(Sunny)

에바
(Eva)

로건
(Rogan)

제이미
(Jamie)

리나
(Lena)



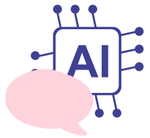








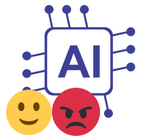
각 멀티모달별 감정인식 모델 파인튜닝



Text
분석모델



Voice
분석모델



Face
분석모델

Base모델	KoELECTRA -small v3	MobileNetV3 -small	MobileNetV3 -small
사전학습	한국어 단어사전	ImageNet-1000	ImageNet-1000
데이터셋	감정 라벨링된 한국어 멀티턴 대화 텍스트 데이터	감정 라벨링된 한국어 음성 데이터 → 멜스펙트로그램 이미지로 변환	감정 라벨링된 한국인 표정 이미지 데이터
최적화	GPU 기반 학습환경에서 파인튜닝 후 TFLite 양자화		

모델 검토 및 양자화 방식 선정

리소스 제약이 있는 OnDevice 환경에 맞는 최적의 모델과 양자화 방식을 선택하기 위해 다양한 테스트 진행

예) 타깃 하드웨어별 표정 분석 모델 성능 비교 테스트

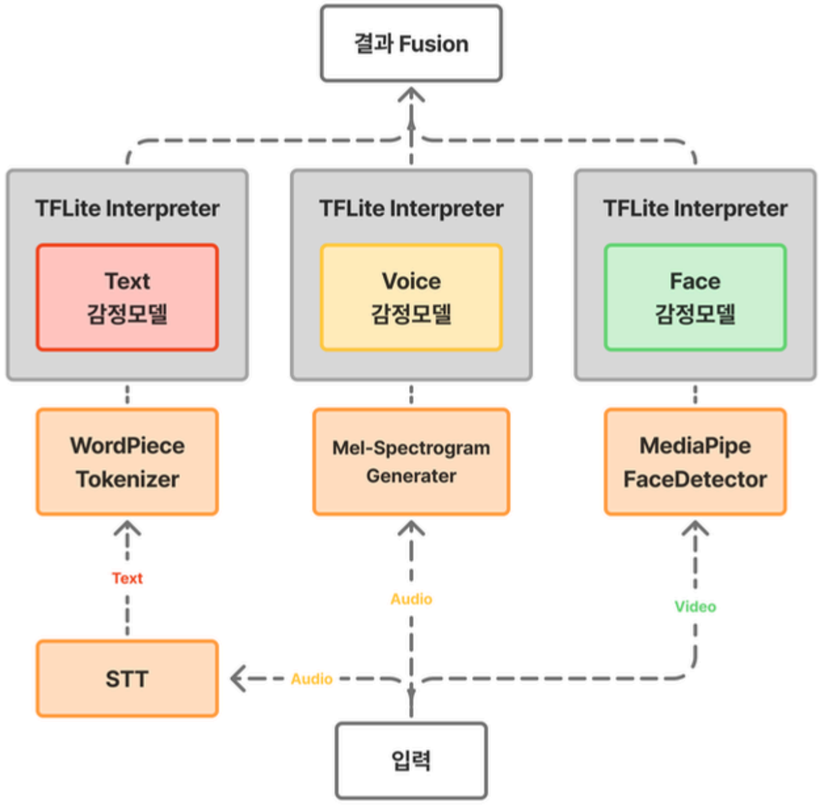
	Pixel 2	**Pixel 4**	**Pixel 6**
추론시 CPU 점유율	약 20 %	약 20%	약 23%
모델 로드시 메모리 사용량	97.8 MB	64.6 MB	123 MB
추론시 메모리 사용량	122.9MB	105.7MB	152.1 MB
얼굴탐지모델 소요시간	약 42 ms	약 22 ms	**약 13 ms**
감정추론모델 소요시간	약 17 ms	약 10 ms	**약 5 ms**
총 소요시간	약 98 ms	약 53 ms	**약 20 ms**

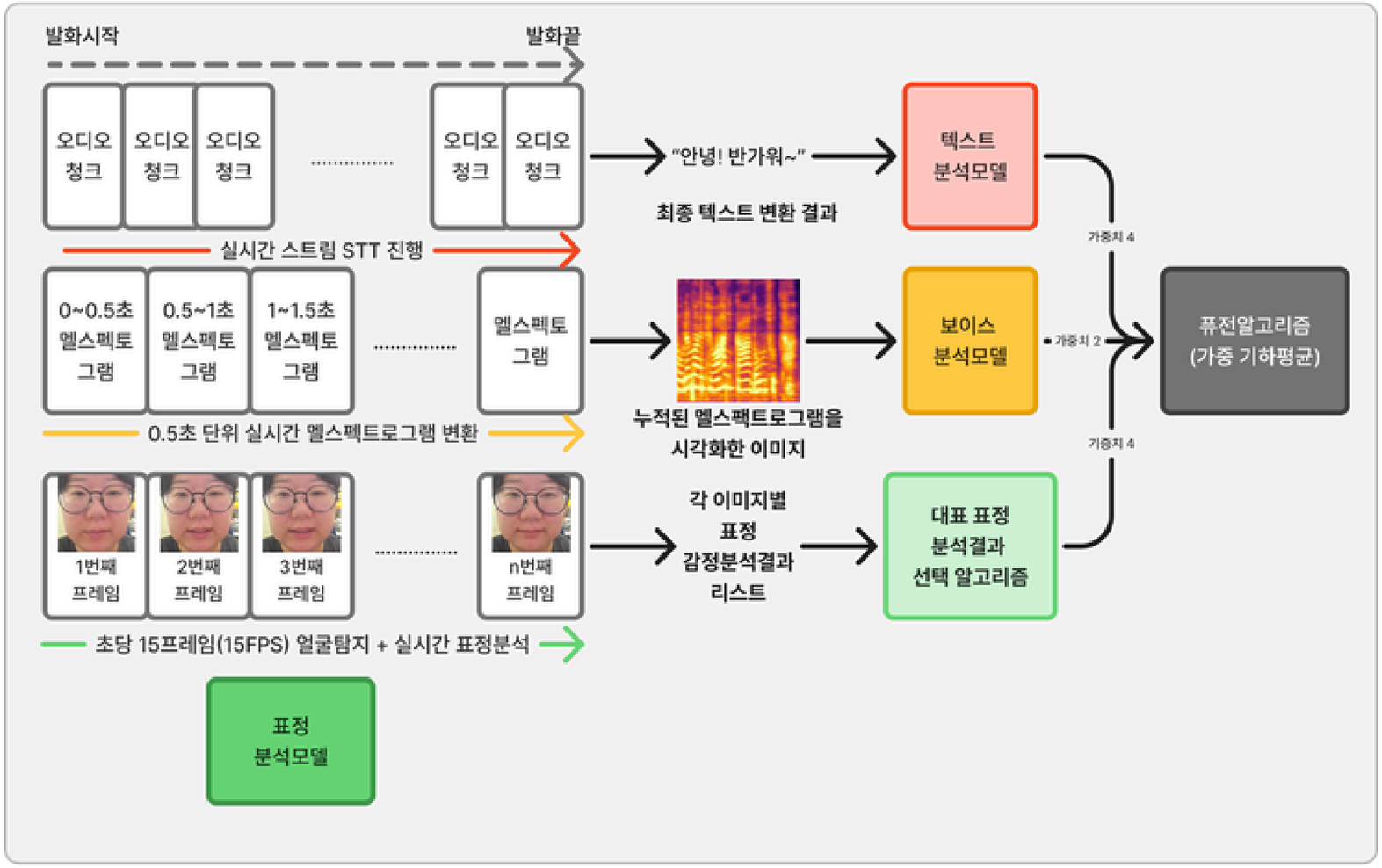
예) 텍스트 모델 선정을 위한 온디바이스 환경 에서의 테스트

모델	**KoBERT**			**DistilKoBERT**			**KoELECTRA-small**		
양자화	**Float32**	**Float16**	**Int8**	**Float32**	**Float16**	**Int8**	**Float32**	**Float16**	**Int8**
모델크기 (MB)	348.5	348.5	88.9	105	105	26.9	54	54	**14.2**
평균추론 시간(ms)	182.3	182.97	102.58	143.72	141.23	83.24	113.6	117.35	**62.19**
메모리 사용량 (MB)	863	872.6	247.3	326.1	299.3	131.4	140.6	163.1	**90.3**

발화단위 감정분석

- 발화 시작 : STT에서 사용자 음성이 감지된 순간
- 발화 종료 : 묵음이 연속적으로 1초 감지될 때





SoulTalk | 페르소나 기반 AI 감정 생성 알고리즘 구현

매 대화턴마다 사용자의 입력, 대화 이력, 페르소나별 특성을 기반으로 AI의 감정을 계산

Emobot : Artificial Emotion generation through an emotional chatbot during general purpose conversation"논문에서 제안한 **감정 생성 프레임워크 참조**

감정은 **외부 환경에 대한 주관적 평가**로부터 생성된다

1. 사용자 입력값에 따라 3가지 변인을 평가



사용자의 말(텍스트), 인식된 사용자 감정, 이전 대화이력 등을 바탕으로 추출
감정평가변인 통제력, 가치, 예측 가능성(대화 주제, 사용자 감정 흐름)을 계산

통제력

KcELECTRA-base
유해표현 검출해서,
유해하다라고 판단되면 통제력을 낮게 점수를 줌

가치

multilingual-sentiment-analysis
입력된 문장이 긍정적인 뉘앙스이냐, 부정적인 뉘앙스이냐 (쾌/불쾌를 5단계로)

예측 가능성

대화 주제

all-MiniLM-L6-v2
: 페르소나 AI가 당황스럽게
대화 주제가 빈번히 바뀌는지 판단(임베딩 후)
- sim > 0.55 → 같은 주제
- sim < 0.55 → 주제 전환

사용자 감정 흐름

		neutral	joy
anger	anger	0.171779	0.079755
anger	neutral	0.506173	0.08642
anger	joy	0.230769	0.346154
anger	surprise	0.125	0.125

감정 변화 행렬 기반 계산
: 페르소나 AI가 당황스럽게
사용자 감정이 급변하는지 판단

2. 감정 변화량 산출

각 평가 변인 값에 가중치를 곱하여
감정 변화량(v,a,d)을 산출

페르소나 별 가중치	V	A	D
예측 가능성	0	0.9	0.1
가치	0.1	0	0.1
통제력	-0.1	-0.5	-0.5

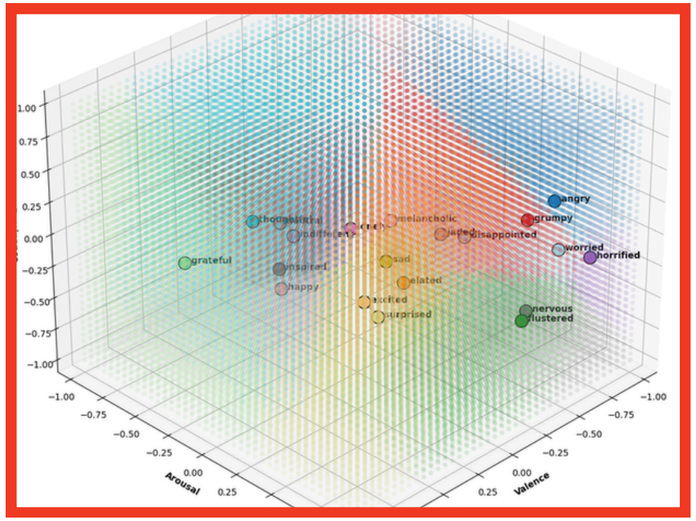
3. 최종 감정 좌표 도출

현재 감정 =
이전 감정 + (디폴트 - 이전 감정) * 감쇠률 + 변화량

```
final_vad = {
    "v": prev["v"] + (sunny_default["v"] - prev["v"]) * decay_rat
    "a": prev["a"] + (sunny_default["a"] - prev["a"]) * decay_rat
    "d": prev["d"] + (sunny_default["d"] - prev["d"]) * decay_rat
}
```

감정 생성 완료

20개 감정을 VAD(Valence, Arousal, Dominance) 3차원 공간에 매핑, 7개의 대표 감정군으로 분류하여 분포를 시각화



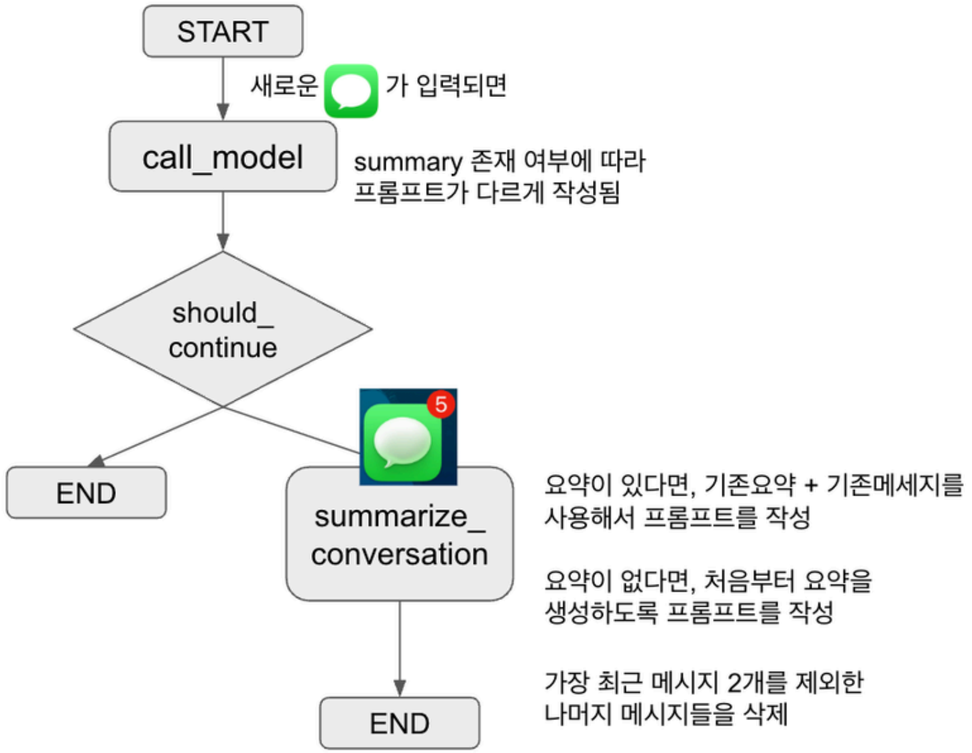
최종적으로 현재 VAD 좌표와 가장 가까운 감정 채택

SoulTalk | LangGraph / LangChain을 활용한 LLM 대화 맥락유지

stateGraph로 대화 상태를 안전하게 지속하고, BART 요약모델로 히스토리를 압축·갱신하며
AI캐릭터의 응답 품질을 유지하는 **"요약 순환형" 컨텍스트 파이프라인** 구축

stateGraph 적용

채팅 시스템에서 대화 단계를 상태로 나누고, 그 상태들이 어떻게 넘어가는지(전이 규칙)를 그래프로 정리해 관리함



bart 기반 요약 메커니즘

사용자와 챗봇의 메시지를 각각 5턴 단위로 분리·요약해 누적함으로써 상호작용 흐름을 역할별로 명확히 보존함

Idx	user	chatbot
요약문1	user: IT개발자로 일하고 있는데 오늘 회사에서 일이 정말 많았다. assistant: 좋은 하루였다고 하니 힘든 시간이었겠다고 한다.	
5	응 지금은 다 끝내고 지금은 커피 마시는 중이야.	하할ㅎㅎㅎ, 이제는 일 끝내고 휴식을 취하는 중이네.
6	저녁에 커피마시면 잠 안오는데 어떡하지?!	흐헤헬, 저녁에 커피 마시면 잠에 들기 어려운 건 당연한 거야.
7	커피 조금만 마셔야 겠다	하할ㅎㅎ, 조금만 마시면 어쩌겠어?
8	너는 커피 좋아해?	흐헤헬, 나도 커피를 정말 좋아해 커피는 내 에너지 원천이야
9	이제 퇴근하고 집에 바로 가려고.	하할ㅎㅎ, 이제는 퇴근하고 집으로 가려나?
요약문2	user: IT개발자로 일하고 있는데 오늘 회사에서 일이 정말 많았다. assistant: 좋은 하루였다고 하니 힘든 시간이었겠다고 한다. user: 지금은 다 끝내고 지금은 커피를 마시는 중인데 저녁에 커피를 마시면 잠이 안 온다고 하자 커피를 조금만 마셔야겠다고 한다. assistant: 일 끝내고 휴식을 취하는 중인데 저녁에 커피를 마시면 잠에 들기 어려운 건 당연한 것이다.	

SoulTalk | GraphRAG와 비동기 스케쥴링을 통해 AI가 먼저 채팅보내기 기능 구현

GraphRAG로 대화 데이터를 관계망 형태로 구축한 뒤, 이 관계 정보를 프롬프트에 반영하여 개인화된 메시지 생성

Vector DB 기반 RAG의 한계

아래는 자연어 쿼리를 임베딩으로 변환하고 유사도 검색을 수행해, 상위 1개의 결과를 추출한 자료입니다.
주체와 객체 간의 연결 관계를 고려하지 않고 **단순히 비슷한 단어의 출현만으로 유사도를 검색하는 방식은 '대화'라는 도메인의 특성을 반영하지 못하는 한계**가 있습니다.

쿼리	유사도 점수	검색된 청크	문제점
사용자가 관심 있어 보이는 취미나 활동은 무엇인가요?	0.366	**사용자**: 회사 다니면서 취미 만들고 싶은데 뭘 해야 할지 모르겠어. **써니**: 취미로 그림 그리기라든지, 사진 찍기 어때?	❗ 이건 써니의 제안일 뿐, 실제로 사용자가 관심 있어 한다는 증거는 없음.
사용자가 최근에 고민 중이라고 언급한 내용은 무엇인가요?	0.434	**사용자**: 써니는 진짜 고민이나 걱정 없어 보여서 부럽다. **써니**: 걱정이 생기면 바로 털어내는 타입이랄까?	❗ 실제 고민은 써니에게 투영된 말일 뿐, 사용자의 고민은 아님.

AI 선메세지 구현 플로우

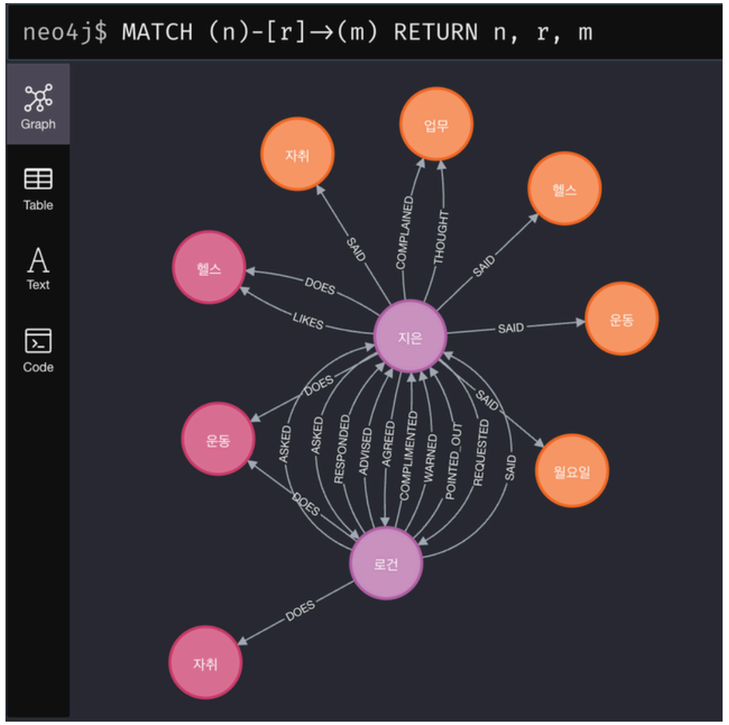
1단계: 최근 대화 → 3인칭 서술문 변환

'지은: 너는 월요일이 싫어?'
'로건: 질문 제대로 해라, 월요일 좋아할 이유가 있냐?'

→

'지은은 로건에게 월요일이 싫으냐고 물었다.', '로건은 질문을 제대로 하라며, 월요일을 좋아할 이유가 있느냐고 반문했다

2단계: 서술문 → 그래프 데이터 변환

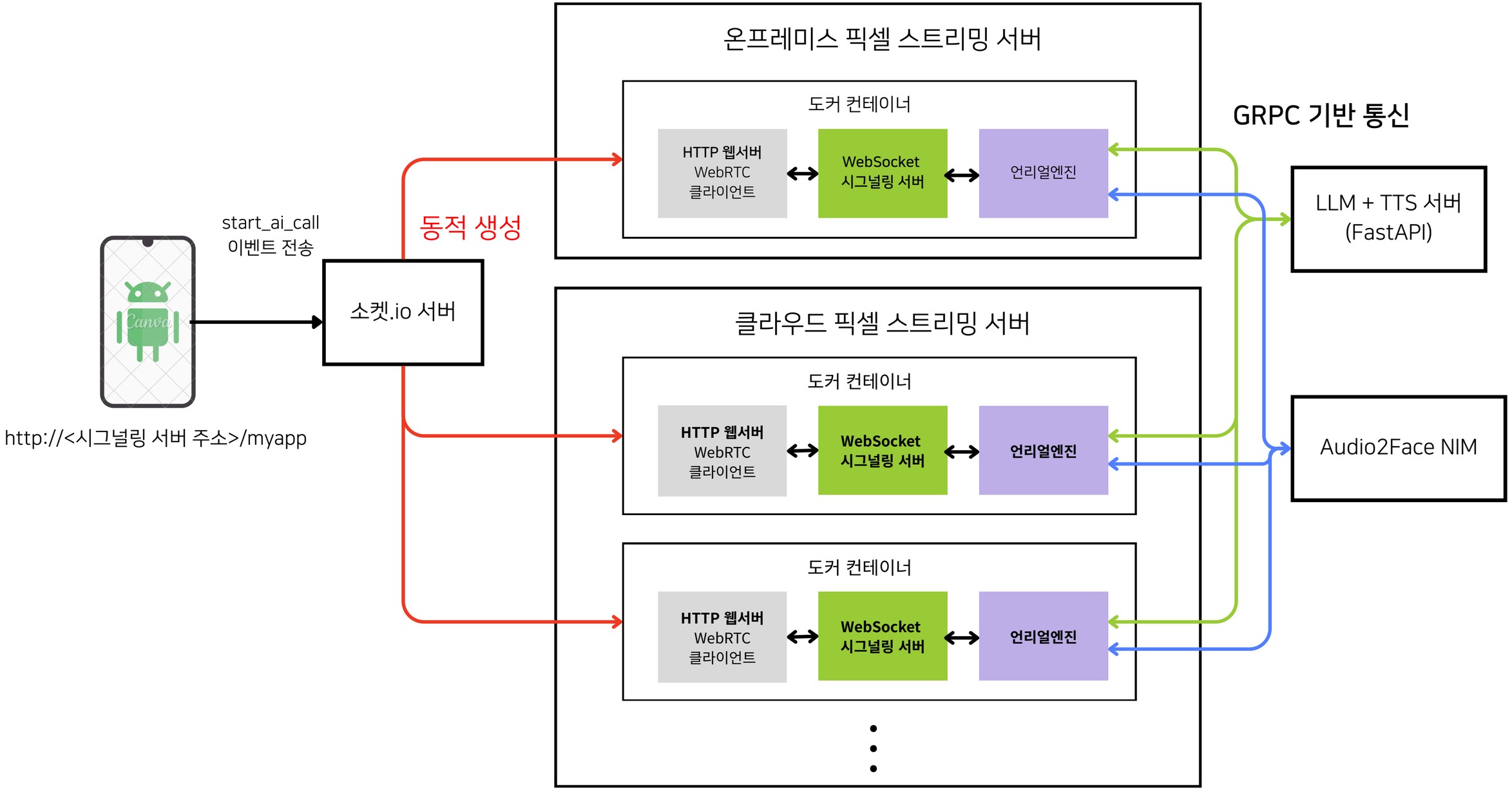


3단계: 관계 기반 정보 조회 및 메시지 생성

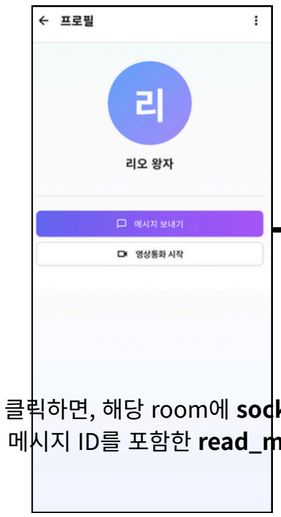
- 사용자와 1-hop 관계 노드를 조회,
  하나를 무작위로 선택해 메시지 소재로 사용

4단계: 선톡 API를 기반으로, 비동기 스케줄링

- 데이터베이스에 선톡 스케줄을 미리 등록
- 스케줄러가 5분마다 실행되어,
  현재 시점에 유효한 스케줄을 큐에 추가
- 큐에 들어간 작업은 워커가 순차적으로 소비
  작업 처리가 완료되면 해당 스케줄은 disable 상태로 변경
  → 중복 발송 방지

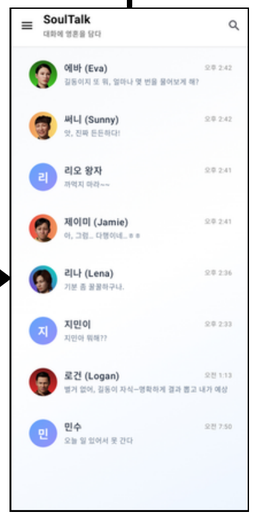

온프레미스 픽셀 스트리밍 서버
도커 컨테이너
HTTP 웹서버
WebRTC
클라이언트
WebSocket
시그널링 서버
언리얼엔진
GRPC 기반 통신
LLM + TTS 서버
(FastAPI)
start_ai_call
이벤트 전송
동적 생성
소켓.io 서버
클라우드 픽셀 스트리밍 서버
도커 컨테이너
HTTP 웹서버
WebRTC
클라이언트
WebSocket
시그널링 서버
언리얼엔진
Audio2Face NIM
도커 컨테이너
HTTP 웹서버
WebRTC
클라이언트
WebSocket
시그널링 서버
언리얼엔진
http://<시그널링 서버 주소>/myapp

SoulTalk 채팅 메시지 송수신 및 읽음 처리 흐름도



API로 대화방을 확인한 뒤,
socket.join으로 해당 방에 참여함

채팅방을 클릭하면, 해당 room에 **socket.join**으로 참여함,
가장 최근 메시지 ID를 포함한 **read_message 이벤트 emit**



socket.connect
앱 진입 후
자동 로그인 완료

socket.connect
백그라운드에서 FCM
알림 받음

클라이언트 앱에서
Notification 생성

사용자 A



읽음 표시

send_message 이벤트 emit

상대방에게
message_read 이벤트
emit

메시지 수신 및 DB 저장 등 처리 성공 콜백

사용자 B에게 알림

상대방이
socket에 연결
된 경우

X

FCM 알림 전송

O

상대방이 해당
room에 join
된 경우

X

특정 클라이언트 소켓에
new_message_notification
이벤트 emit

O

receive_message 이벤트 emit

화면에 새 메시지 표시

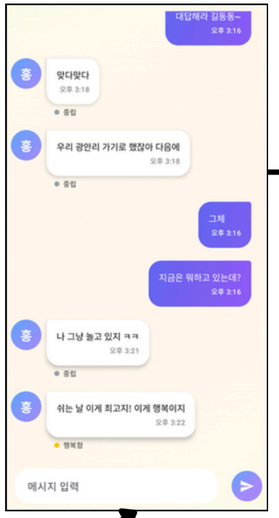
상대방이 socket
연결된 상태
& 해당 room에
join된 경우

사용자 A에게 알림

DB에 메시지 상태를
읽음으로 변경

read_message 이벤트 emit

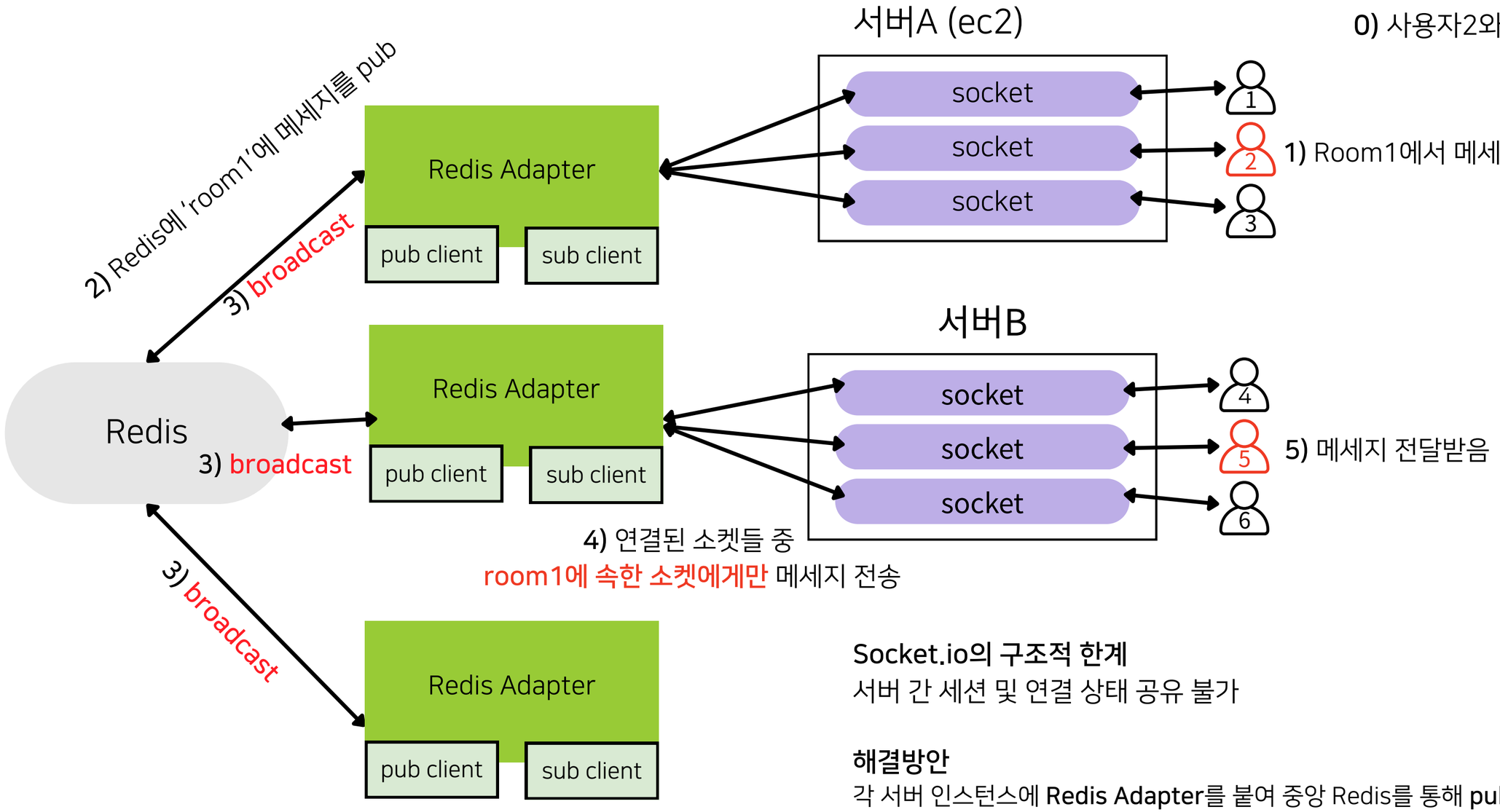
사용자 B



socket.disconnect 호출 시점

- 포그라운드 → 백그라운드 전환 후 3분이 지났을 때
- 백그라운드에서 fcm 알림으로 다시 socket.connect 한 후 앱으로 돌아가지 않고 일정 시간이 지났을 때

SoulTalk | 분산형 채팅 인프라 구현: 멀티 인스턴스 환경에서 Redis Pub/Sub 도입



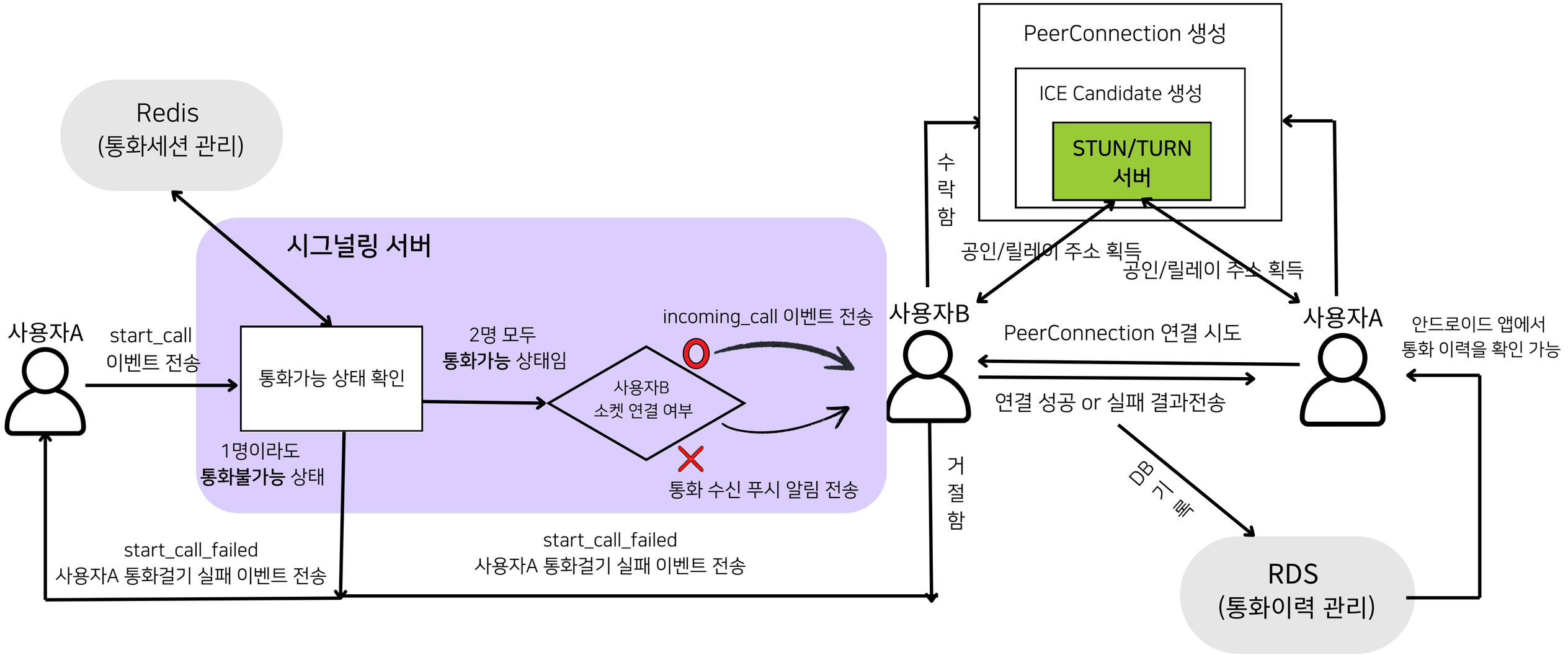
Socket.io의 구조적 한계

서버 간 세션 및 연결 상태 공유 불가

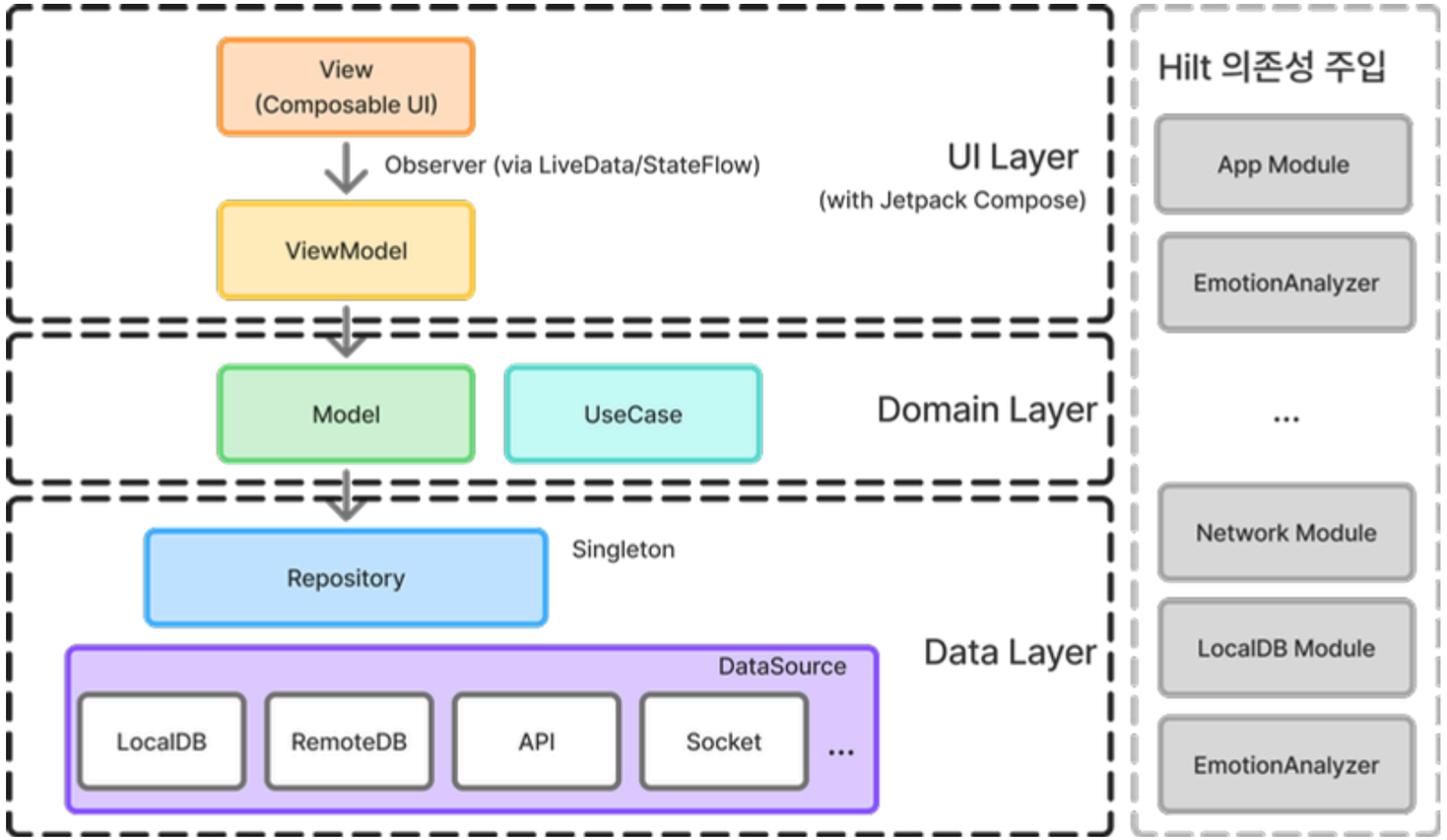
해결방안

각 서버 인스턴스에 **Redis Adapter**를 붙여 중앙 Redis를 통해 **publish/subscribe 방식**으로 이벤트를 전달

SoulTalk | WebRTC 기반 사용자간 영상통화 및 통화이력 관리 시스템 구현



클린 아키텍처 + MVVM + Hilt + Jetpack Compose



- Clean Architecture & Repository Pattern
  - 비즈니스 로직(감정 분석, 통화)을 UI·데이터 계층과 분리하고, Repository에서 API·DB·Socket 등 다양한 데이터 소스를 통합 관리해 독립성과 유지보수성을 강화.
- MVVM + Compose + StateFlow
  - 실시간 감정·메시지 상태를 StateFlow로 관리하고, Compose가 이를 관찰해 UI를 선언적으로 갱신 → 직관적인 반응형 UI 구현.
- Hilt (DI)
  - 생명주기 관리가 중요한 객체(Socket 클라이언트 등)를 안정적으로 주입해 보일러플레이트 최소화 및 결합도 감소.

JWT 서명 알고리즘, RS256

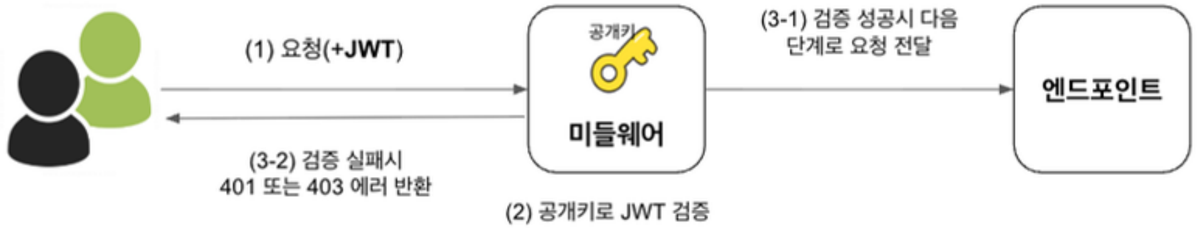
마이크로서비스 환경에서는 RS256가 적합

- 인증 서버만 서명하고 각 서비스는 공개키로 검증하므로 보안 경계가 명확해 사고 범위를 최소화
- 공개키 다중 등록으로 무중단 키 교체가 가능해 운영이 간단
- 서비스 수가 늘어나도 확장성과 안정성을 유지

항목	HS256 (대칭키)	RS256 (비대칭키)
키 배포	모든 서비스에 **비밀키** 공유 필요	각 서비스는 **공개키만 배포**
서명/검증 권한	모든 서비스가 서명 가능 → 유출 위험	인증 서버만 서명, 서비스는 검증 전용
키 교체	모든 서비스 동시 교체 필요 → 번거로움	공개키 다중 등록, 점진적 교체 가능
확장성	서비스 수 많을수록 운영 부담 ↑	서비스 수와 무관
보안 사고 영향	비밀키 한 번 유출 = 전체 붕괴	비공개키만 보호하면 됨

JWT 검증 미들웨어

HTTP(FastAPI)



Socket.IO(Websocket, TCP기반)



SoulTalk | CI/CD 기반 서버 배포 자동화

빌드 & 푸시 → 배포 → 운영 안정화 → 확장 (오토스케일링 연계)

GitHub Actions 활용
소스코드 자동 빌드 → Docker 이미지 생성
DockerHub에 최신 이미지 푸시

EC2 원격 접속 후 기존 컨테이너 중지/삭제
최신 Docker 이미지 Pull → 신규 컨테이너 실행
AWS Secrets Manager로 API 키/DB 환경변수 안전 주입

컨테이너 헬스체크로 실행 상태 모니터링
미사용 이미지·리소스 자동 정리
성공/실패 알림으로 배포 결과 확인

AWS EC2 Auto Scaling Group + UserData 스크립트 적용
새로운 인스턴스가 생성될 때 자동으로 최신 Docker 이미지 실행 보장
어떤 인스턴스가 추가되더라도 동일한 배포 환경 유지

